

PYTHON • LISTAS

Métodos de Listas en Python

`append` · `extend` · `insert` · `remove` · `pop` · `index` · `count` · `sort` · `reverse` · `copy` · `clear`

3

1

4

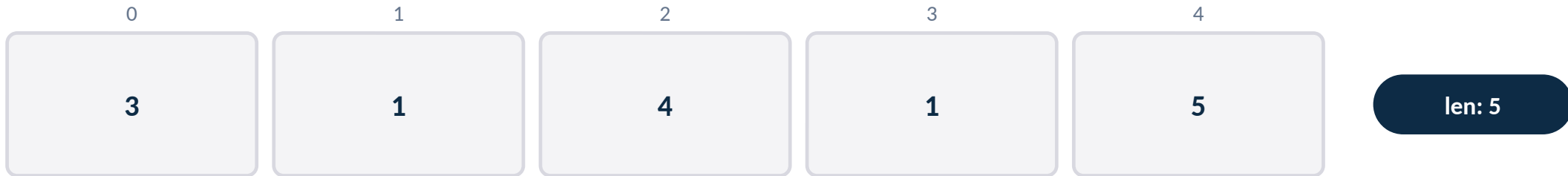
1

5

9

La lista con la que vamos a trabajar

```
numeros = [3, 1, 4, 1, 5]
```



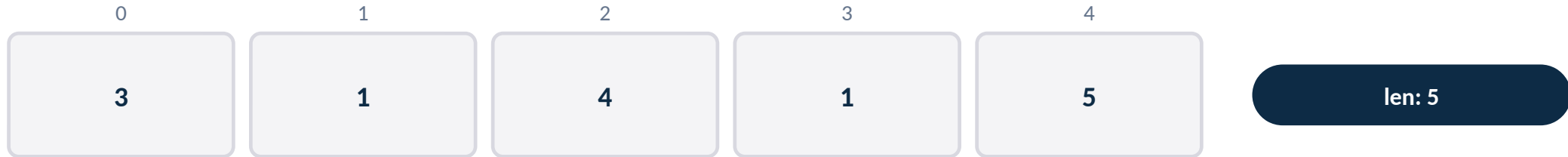
- Una lista en Python guarda una secuencia ordenada de valores, cada uno con un índice que empieza en 0.
- `numeros` tiene 5 elementos: índices del 0 al 4, y `len(numeros) = 5`. Fíjate que el valor 1 aparece dos veces (índices 1 y 3) — nos va a servir más adelante.
- En las próximas diapositivas vamos a aplicar, en orden, los 11 métodos de la tabla sobre esta misma lista, viendo cómo cambia su contenido y su longitud paso a paso.

append(x) — agrega al final

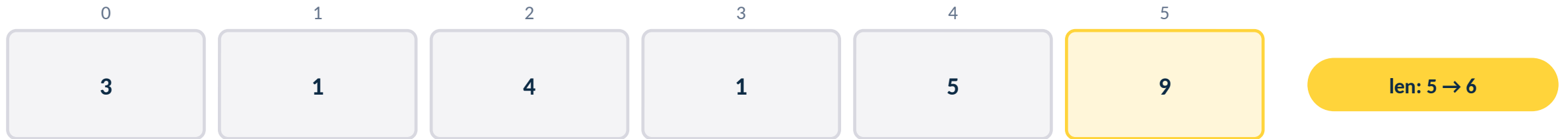
```
numeros.append(9)
```

- Agrega UN elemento al final de la lista.
- Modifica (muta) la lista original.
- No devuelve nada (devuelve None).

ANTES



DESPUÉS DE APPEND(9)



append() siempre agrega UN solo elemento, aunque ese elemento sea a su vez una lista.

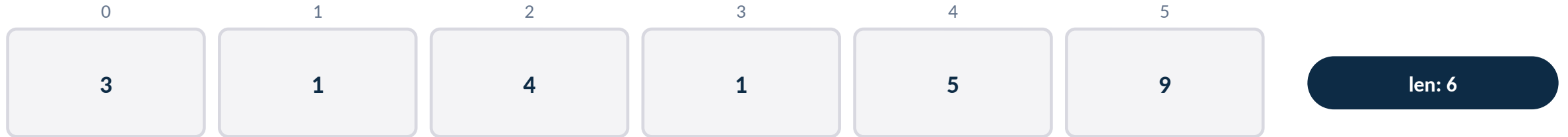
MÉTODO 2 DE 11

extend(iter) — agrega varios al final

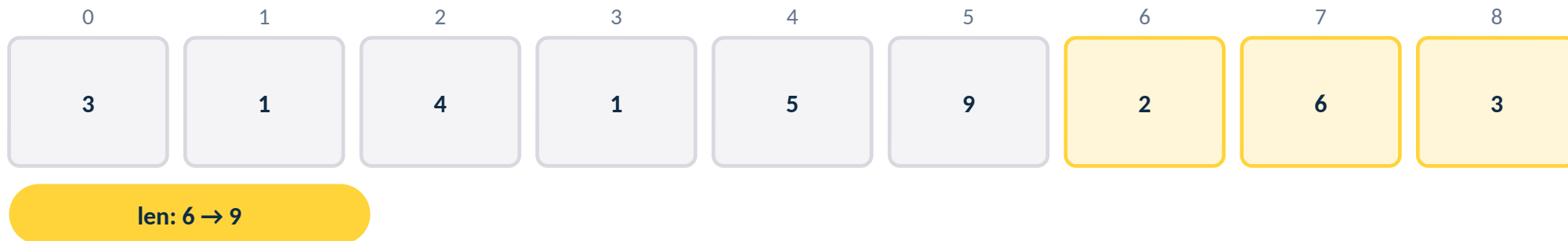
```
numeros.extend([2, 6, 3])
```

- Agrega CADA elemento del iterable al final, uno por uno.
- Modifica (muta) la lista original.
- No es lo mismo que append(): extend() "desarma" el iterable.

ANTES



DESPUÉS DE EXTEND([2, 6, 3])



numeros.append([2,6,3]) habría agregado UNA lista adentro (len 7); extend() agrega los 3 valores sueltos (len 9).

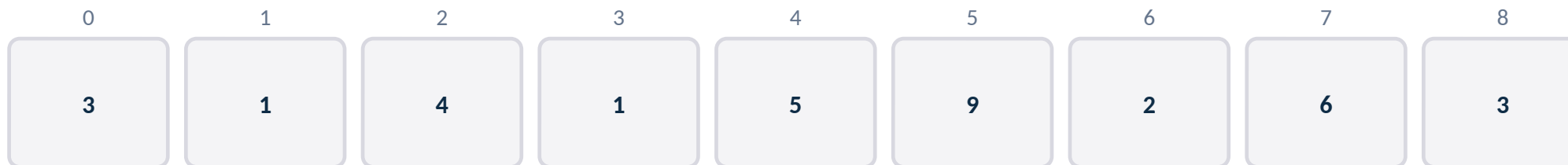
MÉTODO 3 DE 11

insert(i, x) — inserta en una posición

```
numeros.insert(0, 7)
```

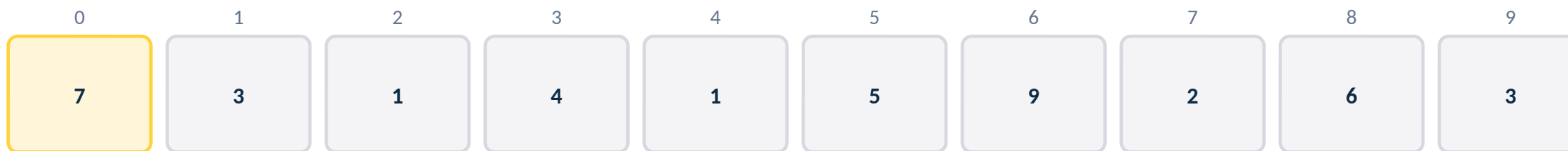
- Inserta x ANTES del índice i indicado.
- Los elementos desde esa posición se corren hacia la derecha.
- Modifica (muta) la lista original.

ANTES



len: 9

DESPUÉS DE INSERT(0, 7)



len: 9 → 10

7 entra en el índice 0. Todo lo demás (3, 1, 4, 1, 5, 9, 2, 6, 3) se corre un lugar hacia la derecha.

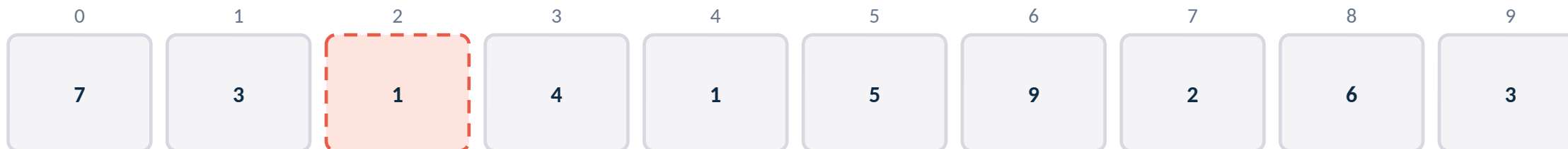
MÉTODO 4 DE 11

remove(x) — elimina la primera ocurrencia

```
numeros.remove(1)
```

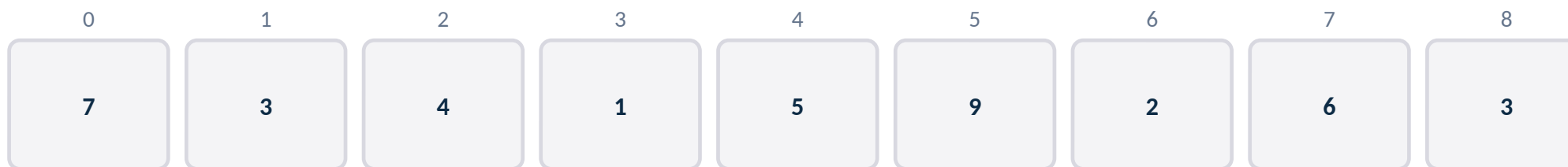
- Busca el valor x y elimina SOLO su primera aparición.
- Si x aparece varias veces, las demás quedan intactas.
- Si x no está en la lista, lanza un ValueError.

ANTES



len: 10

DESPUÉS DE REMOVE(1)



len: 10 → 9

Se elimina el 1 del índice 2 (el PRIMERO que aparece). El otro 1 (más adelante) queda en la lista.

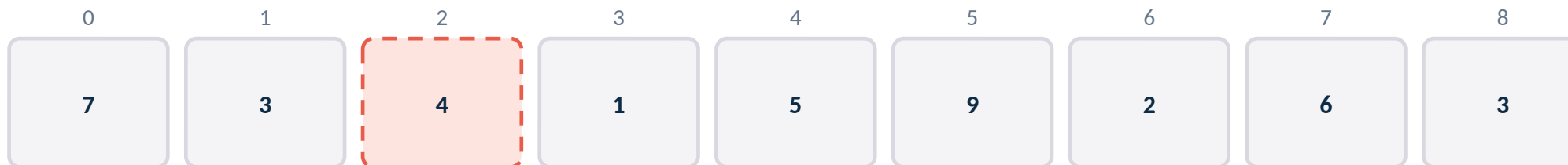
MÉTODO 5 DE 11

pop(i) — elimina y devuelve por posición

```
eliminado = numeros.pop(2)
```

- Elimina el elemento en la posición i Y lo devuelve.
- Sin argumento, pop() elimina y devuelve el ÚLTIMO elemento.
- Modifica (muta) la lista original.

ANTES



len: 9

DESPUÉS DE POP(2)



len: 9 → 8

eliminado vale 4 (lo que estaba en el índice 2). A diferencia de remove(), pop() SÍ devuelve el valor sacado.

index(x) — busca la primera posición

```
numeros.index(9)
```

- Devuelve el índice de la PRIMERA aparición de x.
- NO modifica la lista (no muta).
- Si x no está en la lista, lanza un ValueError.

NUMEROS (NO CAMBIA)



len: 8

numeros.index(9) → 4

El 9 está en el índice 4. La lista queda exactamente igual que antes: index() solo consulta.

count(x) — cuenta apariciones

```
numeros.count(3)
```

- Devuelve CUÁNTAS veces aparece x en la lista.
- NO modifica la lista (no muta).
- Si x no está, devuelve 0 (no lanza error).

NUMEROS (NO CAMBIA)



len: 8

numeros.count(3) → 2

sort() — ordena de menor a mayor

```
numeros.sort()
```

- Ordena la lista en el lugar, de menor a mayor.
- Modifica (muta) la lista original — no devuelve una nueva.
- Para orden descendente: `numeros.sort(reverse=True)`.

ANTES



len: 8

DESPUÉS DE SORT()



len: 8 (no cambia)

El len() no cambia con sort(): son los MISMOS 8 elementos, solo que reordenados.

reverse() — invierte el orden

```
numeros.reverse()
```

- Invierte el orden actual de la lista, en el lugar.
- NO ordena — solo da vuelta lo que ya había.
- Modifica (muta) la lista original.

ANTES



len: 8

DESPUÉS DE REVERSE()



len: 8 (no cambia)

Como la lista ya estaba ordenada ascendente, reverse() la deja en orden descendente — pero no es lo mismo que sort(reverse=True) en listas no ordenadas.

copy() — devuelve una copia independiente

```
nueva = numeros.copy()  
nueva.append(100)
```

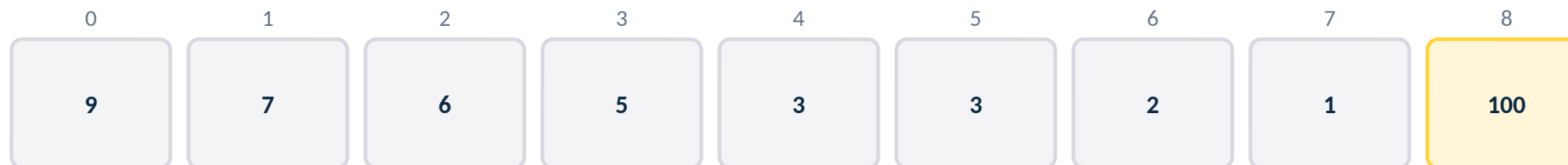
- Crea una copia superficial (shallow) e independiente.
- NO modifica numeros — devuelve una lista nueva.
- Modificar nueva después NO afecta a numeros (y viceversa).

NUMEROS (NO CAMBIA)



len: 8

NUEVA, DESPUÉS DE NUEVA.APPEND(100)



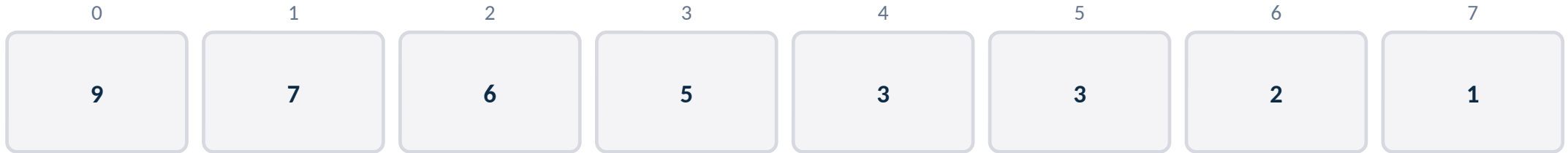
len: 9

numeros sigue midiendo 8 — el 100 solo se agregó en nueva, la copia independiente.

clear() — vacía la lista

```
numeros.clear()
```

ANTES



len: 8

DESPUÉS DE CLEAR()

[] vacía

len: 8 → 0

- Elimina TODOS los elementos de la lista.
- Modifica (muta) la lista original: no crea una nueva vacía.
- Equivale a hacer `numeros[:] = []` o del `numeros[:]`.

numeros pasa a ser [] pero sigue siendo la MISMA lista en memoria, ahora vacía — no es una lista nueva.

RESUMEN

Los once métodos, de un vistazo

Método	¿Muta?	¿Devuelve algo útil?
<code>append(x)</code>	Sí	No (None)
<code>extend(iter)</code>	Sí	No (None)
<code>insert(i, x)</code>	Sí	No (None)
<code>remove(x)</code>	Sí	No (None)
<code>pop([i])</code>	Sí	Sí — el elemento eliminado
<code>index(x)</code>	No	Sí — el índice encontrado
<code>count(x)</code>	No	Sí — la cantidad de apariciones
<code>sort()</code>	Sí	No (None)
<code>reverse()</code>	Sí	No (None)
<code>copy()</code>	No	Sí — una lista nueva e independiente
<code>clear()</code>	Sí	No (None)

index() y count() son las únicas que solo consultan; el resto de los que aparecen acá mutan la lista original.

PARA RECORDAR

¿Muta la lista?

¿Devuelve algo útil? Con eso alcanza.

- `append` / `extend` / `insert` agregan elementos; `remove` / `pop` / `clear` los sacan.
- `pop()` es el único que elimina Y devuelve el valor sacado — `remove()` no devuelve nada.
- `index()` y `count()` son de solo consulta: nunca modifican la lista.
- `sort()` y `reverse()` reordenan en el lugar — el `len()` nunca cambia con ellos.
- `copy()` es la excepción para duplicar: devuelve una lista nueva e independiente.